

HA Kubernetes Lab: Cilium, BGP & OPNsense

2026

Author: Rodríguez Rubén

INDICE:

INTRODUCTION.....	3
Step 1: Automated OS Bootstrapping & Node Preparation.....	5
1.1. Install and Configure Ansible (Only on k8s-control-01).....	5
1.2. Create the Encrypted Ansible Inventory & Credentials.....	6
1.3. Verify Connectivity & Vault Authentication (Ping Test).....	6
1.4. Base OS & Runtime Preparation Playbook (setup.yml).....	7
1.5 Playbook Execution & Results.....	8
Step 2: Kubernetes Toolchain Installation & Control Plane Initialization	9
2.1. Kubernetes Toolchain Playbook (k8s-pkg.yml).....	9
2.2. Control Plane Initialization & Worker Join Playbook (k8s-init.yml)	11
2.3. CNI (Container Network Interface) Deployment.....	13
Step 3: Cilium BGP Dynamic Routing & Edge Integration.....	14
Step 4. Deploying Test Application & LoadBalancer IP Announcement.....	18
CONCLUSION:.....	20

INTRODUCTION

Modern cloud environments move fast, and traditional networking models can't always keep up. Static routes and manual firewall rules quickly become bottlenecks when you're dealing with containerized workloads that spin up and down in milliseconds.

The goal of this project was to build a production-grade, highly available Kubernetes infrastructure from scratch. Using **Cilium (eBPF)** for kernel-level networking and **BGP** for dynamic routing, the cluster communicates directly with an **OPNsense** edge firewall. When a service deploys or migrates, network routes update automatically across the environment—no human intervention required.

This lab documents the architecture, implementation, and real-world failure testing used to validate automatic failover and zero-downtime resilience.

Important: Basic operating system installation, virtual machine provisioning, and standard initial OS configurations are omitted from this documentation to focus strictly on network architecture, eBPF/Cilium integration, BGP dynamic routing, and high-availability validation

Step 0: Prerequisites & Environment

Virtual Machine	Role / Function	OS / Base Image	vCPU	RAM	Storage	Network Interfaces
gateway-01	Edge Router / BGP Peer	OPNsense 24.x	2	2 GB	10 GB	VMnet8 (WAN) / VMnet1 (LAN)
k8s-control-01	Kubernetes Control Plane	Ubuntu 26.04 LTS	2	4 GB	20 GB	VMnet1 (LAN)
k8s-worker-01	Worker Node 1 (Cilium CNI)	Ubuntu 26.04 LTS	2	4 GB	20 GB	VMnet1 (LAN)
k8s-worker-02	Worker Node 2 (Cilium CNI)	Ubuntu 26.04 LTS	2	4 GB	20 GB	VMnet1 (LAN)
Total Resources	Host Allocation	—	8 Cores	16 GB	70 GB	—

Step 1: Automated OS Bootstrapping & Node Preparation

In this phase, we prepare the base operating system across all three nodes using **Ansible**, executed directly from k8s-control-01.

1.1. Install and Configure Ansible (Only on k8s-control-01)

Install Ansible on the control plane node to orchestrate the configuration across the entire cluster:

```
ruben@control:~$ ssh-copy-id ruben@10.10.10.40
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/ruben/.ssh/id_rsa
.pub"
The authenticity of host '10.10.10.40 (10.10.10.40)' can't be established.
ED25519 key fingerprint is: SHA256:5S6zo7isamo2c5uj25krfJT8Swcn5ABZKD0uTABHmw
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out an
y that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now
it is to install the new keys
ruben@10.10.10.40's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'ruben@10.10.10.40'"
and check to make sure that only the key(s) you wanted were added.

ruben@control:~$ ssh-copy-id ruben@10.10.10.4
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/ruben/.ssh/id_rsa
.pub"
The authenticity of host '10.10.10.4 (10.10.10.4)' can't be established.
ED25519 key fingerprint is: SHA256:5S6zo7isamo2c5uj25krfJT8Swcn5ABZKD0uTABHmw
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out an
y that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now
it is to install the new keys
ruben@10.10.10.4's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'ruben@10.10.10.4'"
and check to make sure that only the key(s) you wanted were added.

ruben@control:~$
```

1.2. Create the Encrypted Ansible Inventory & Credentials

To avoid hardcoding sensitive credentials in plain text, store the sudo password using **Ansible Vault** and reference it securely in your variables.

Step A: Create an Encrypted Vault File

On k8s-control-01, create a vault file to store the sensitive variable and then, we create the inventory.

```
GNU nano 8.7.1 inventory.ini
[k8s_nodes]
10.10.10.4 ansible_user=ruben
10.10.10.40 ansible_user=ruben
[k8s_nodes:vars]
ansible_python_interpreter=/usr/bin/python3
```

1.3. Verify Connectivity & Vault Authentication (Ping Test)

Before running OS-level modifications, verify SSH reachability and privilege escalation (sudo) across all inventory targets using the Vault-encrypted credentials:

```
ruben@control:~/project$ ansible k8s_nodes -m ping -i inventory.ini --ask-vault-pass
Vault password:
10.10.10.4 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
10.10.10.40 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
ruben@control:~/project$
```

1.4. Base OS & Runtime Preparation Playbook (setup.yml)

Create the playbook setup.yml to automate disabling SWAP, loading kernel modules required by Cilium/Kubernetes, tuning sysctl net-forwarding parameters, and configuring containerd as the Container Runtime Interface (CRI):

On Ubuntu 26.04, Ansible requires specifying the ansible_become_exe path due to underlying changes in how sudo execution wrappers function.

```
[k8s_nodes:vars]
ansible_python_interpreter=/usr/bin/python3
ansible_become_exe=/usr/bin/sudo.ws
```

Create setup.yml to automate disabling SWAP, loading eBPF/K8s kernel modules, tuning sysctl parameters, and initializing containerd as the Container Runtime Interface (CRI):

```
ruben@control:~/project$ cat setup.yml
---
- name: Prepare OS and Runtime for Kubernetes & Cilium
  hosts: k8s_nodes
  become: true
  tasks:
    - name: Disable SWAP in memory
      command: swapoff -a
      when: ansible_facts['swaptotal_mb'] > 0

    - name: Disable SWAP in /etc/fstab
      replace:
        path: /etc/fstab
        regexp: '^(^#\s).*?\sswap\s+sw\s+.*$'
        replace: '# \1'

    - name: Create k8s modules load configuration
      copy:
        dest: /etc/modules-load.d/k8s.conf
        content: |
          overlay
          br_netfilter

    - name: Load Kernel modules
      modprobe:
        name: "{{ item }}"
        state: present
      loop:
        - overlay
        - br_netfilter

    - name: Configure sysctl parameters for Kubernetes
      copy:
        dest: /etc/sysctl.d/k8s.conf
        content: |
          net.bridge.bridge-nf-call-iptables = 1
          net.bridge.bridge-nf-call-ip6tables = 1
          net.ipv4.ip_forward = 1
```


1.5 Playbook Execution & Results

Part of the execution of the playbook.

Execution Summary:

```
ruben@control:~/project$ ansible-playbook -i inventory.ini k8s-pkg.yml --ask-vault-pass
Vault password:

PLAY [Install Kubernetes Binaries] *****

TASK [Gathering Facts] *****
ok: [127.0.0.1]
ok: [10.10.10.4]
ok: [10.10.10.40]

TASK [Install APT prerequisites] *****
ok: [10.10.10.40]
ok: [10.10.10.4]
changed: [127.0.0.1]

TASK [Ensure keyrings directory exists] *****
ok: [127.0.0.1]
ok: [10.10.10.40]
ok: [10.10.10.4]

TASK [Add Kubernetes APT repository GPG key] *****
changed: [127.0.0.1]
ok: [10.10.10.4]
ok: [10.10.10.40]

TASK [Add Kubernetes APT repository] *****
ok: [10.10.10.40]
ok: [10.10.10.4]
changed: [127.0.0.1]

TASK [Install kubelet, kubeadm, and kubectl] *****
ok: [10.10.10.40]
ok: [10.10.10.4]
changed: [127.0.0.1]

TASK [Hold Kubernetes packages from automatic upgrades] *****
changed: [127.0.0.1] => (item=kubelet)
ok: [10.10.10.40] => (item=kubelet)
ok: [10.10.10.4] => (item=kubelet)
changed: [127.0.0.1] => (item=kubeadm)
ok: [10.10.10.40] => (item=kubeadm)
ok: [10.10.10.4] => (item=kubeadm)
changed: [127.0.0.1] => (item=kubectrl)
ok: [10.10.10.40] => (item=kubectrl)
ok: [10.10.10.4] => (item=kubectrl)

PLAY RECAP *****
10.10.10.4 : ok=7 changed=0 unreachable=0 failed=0 skipped=
0 rescued=0 ignored=0
10.10.10.40 : ok=7 changed=0 unreachable=0 failed=0 skipped=
0 rescued=0 ignored=0
127.0.0.1 : ok=7 changed=5 unreachable=0 failed=0 skipped=
0 rescued=0 ignored=0
```


Step 2: Kubernetes Toolchain Installation & Control Plane Initialization

In this phase, we install the official Kubernetes packages across all nodes and initialize the Control Plane on k8s-control-01 without kube-proxy.

2.1. Kubernetes Toolchain Playbook (k8s-pkg.yml)

Create the k8s-pkg.yml playbook to configure the official Kubernetes v1.30 APT repository and install kubelet, kubeadm, and kubectl with package holds enabled to prevent unintended automated updates.

```
---
- name: Install Kubernetes Binaries
  hosts: k8s_nodes
  become: true
  tasks:
    - name: Install APT prerequisites
      apt:
        name:
          - apt-transport-https
          - ca-certificates
          - curl
          - gnupg
        state: present

    - name: Ensure keyrings directory exists
      file:
        path: /etc/apt/keyrings
        state: directory
        mode: '0755'

    - name: Add Kubernetes APT repository GPG key
      get_url:
        url: https://pkgs.k8s.io/core/stable/v1.30/deb/Release.key
        dest: /etc/apt/keyrings/kubernetes-apt-keyring.asc
        mode: '0644'

    - name: Add Kubernetes APT repository
      apt_repository:
        repo: 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.asc] https://pkgs.k8s.io/core/stable/v1.30/deb/ /'
        state: present
        filename: kubernetes

    - name: Install kubelet, kubeadm, and kubectl
      apt:
        name:
          - kubelet
          - kubeadm
          - kubectl
        state: present
        update_cache: yes

    - name: Hold Kubernetes packages from automatic upgrades
      dpkg_selections:
        name: "{{ item }}"
        selection: hold
      loop:
        - kubelet
        - kubeadm
        - kubectl
```

```

ruben@control:~/project$ ansible-playbook -i inventory.ini k8s-pkg.yml --ask-vault-pass
Vault password:

PLAY [Install Kubernetes Binaries] *****

TASK [Gathering Facts] *****
ok: [10.10.10.4]
ok: [10.10.10.40]

TASK [Install APT prerequisites] *****
changed: [10.10.10.4]
changed: [10.10.10.40]

TASK [Ensure keyrings directory exists] *****
ok: [10.10.10.40]
ok: [10.10.10.4]

TASK [Add Kubernetes APT repository GPG key] *****
changed: [10.10.10.4]
changed: [10.10.10.40]

TASK [Add Kubernetes APT repository] *****
changed: [10.10.10.4]
changed: [10.10.10.40]

TASK [Install kubelet, kubeadm, and kubectl] *****
changed: [10.10.10.4]
changed: [10.10.10.40]

TASK [Hold Kubernetes packages from automatic upgrades] *****
changed: [10.10.10.4] => (item=kubelet)
changed: [10.10.10.40] => (item=kubelet)
changed: [10.10.10.4] => (item=kubeadm)
changed: [10.10.10.40] => (item=kubeadm)
changed: [10.10.10.4] => (item=kubectl)
changed: [10.10.10.40] => (item=kubectl)

PLAY RECAP *****
10.10.10.4 : ok=7 changed=5 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
10.10.10.40 : ok=7 changed=5 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0

ruben@control:~/project$

```

2.2. Control Plane Initialization & Worker Join Playbook (k8s-init.yml)

Rather than manually initializing the cluster and copying join tokens across nodes, the entire orchestration is automated via k8s-init.yml.

This playbook initializes k8s-control-01 without kube-proxy, sets up user kubectl environment permissions dynamically using ansible_facts, generates the join command in-memory, and joins all worker nodes:

```
ruben@control:~/project$ cat k8s-init.yml
---
- name: Initialize Kubernetes Control Plane
  hosts: control_plane
  become: true
  tasks:
    - name: Initialize cluster with kubeadm (skipping kube-proxy)
      command: >
        kubeadm init
        --pod-network-cidr=10.244.0.0/16
        --skip-phases=addon/kube-proxy
      args:
        creates: /etc/kubernetes/admin.conf

    - name: Ensure .kube directory exists for local user
      file:
        path: "{{ ansible_facts['env']['HOME'] }}/.kube"
        state: directory
        owner: "{{ ansible_facts['env']['SUDO_USER'] | default(ansible_facts['user_id']) }}"
      ) }}"
      group: "{{ ansible_facts['env']['SUDO_USER'] | default(ansible_facts['user_id']) }}"
      ) }}"
      mode: '0755'

    - name: Copy admin.conf to user kubeconfig
      copy:
        src: /etc/kubernetes/admin.conf
        dest: "{{ ansible_facts['env']['HOME'] }}/.kube/config"
        remote_src: yes
        owner: "{{ ansible_facts['env']['SUDO_USER'] | default(ansible_facts['user_id']) }}"
      ) }}"
      group: "{{ ansible_facts['env']['SUDO_USER'] | default(ansible_facts['user_id']) }}"
      ) }}"
      mode: '0600'

    - name: Generate join command for worker nodes
      command: kubeadm token create --print-join-command
      register: join_command_raw

    - name: Set join command fact
      set_fact:
        join_command: "{{ join_command_raw.stdout }}"

- name: Join Worker Nodes to Cluster
  hosts: workers
  become: true
  tasks:
    - name: Join node to the cluster
      command: "{{ hostvars[groups['control_plane'][0]]['join_command'] }}"
      args:
        creates: /etc/kubernetes/kubelet.conf
ruben@control:~/project$
```

Successful execution of the k8s-init.yml Ansible playbook. The control plane node (127.0.0.1) is initialized first, generating the cluster join token. Subsequently, worker nodes (10.10.10.4 and 10.10.10.40) are automatically joined to the cluster.

```
ruben@control:~/project$ ansible-playbook -i inventory.ini k8s-init.yml --ask-vault-pass
Vault password:

PLAY [Initialize Kubernetes Control Plane] *****

TASK [Gathering Facts] *****
ok: [127.0.0.1]

TASK [Initialize cluster with kubeadm (skipping kube-proxy)] *****
ok: [127.0.0.1]

TASK [Ensure .kube directory exists for local user] *****
ok: [127.0.0.1]

TASK [Copy admin.conf to user kubeconfig] *****
ok: [127.0.0.1]

TASK [Generate join command for worker nodes] *****
changed: [127.0.0.1]

TASK [Set join command fact] *****
ok: [127.0.0.1]

PLAY [Join Worker Nodes to Cluster] *****

TASK [Gathering Facts] *****
ok: [10.10.10.4]
ok: [10.10.10.40]

TASK [Join node to the cluster] *****
changed: [10.10.10.4]
changed: [10.10.10.40]

PLAY RECAP *****
10.10.10.4      : ok=2   changed=1   unreachable=0   failed=0   skipped=0
0   rescued=0   ignored=0
10.10.10.40    : ok=2   changed=1   unreachable=0   failed=0   skipped=0
0   rescued=0   ignored=0
127.0.0.1      : ok=6   changed=1   unreachable=0   failed=0   skipped=0
0   rescued=0   ignored=0
```

Running kubectl get nodes failed with a connection refused error (defaulting to localhost:8080) because the local user lacked the necessary kubeconfig file to authenticate with the Kubernetes API server

```
ruben@control:~/project$ kubectl get nodes -o wide
E0729 14:14:28.416305    90153 memcache.go:265] couldn't get current server API group li
st: Get "http://localhost:8080/api?timeout=32s": dial tcp 127.0.0.1:8080: connect: conn
ection refused
E0729 14:14:28.416661    90153 memcache.go:265] couldn't get current server API group li
st: Get "http://localhost:8080/api?timeout=32s": dial tcp 127.0.0.1:8080: connect: conn
ection refused
E0729 14:14:28.417848    90153 memcache.go:265] couldn't get current server API group li
st: Get "http://localhost:8080/api?timeout=32s": dial tcp 127.0.0.1:8080: connect: conn
ection refused
E0729 14:14:28.418205    90153 memcache.go:265] couldn't get current server API group li
st: Get "http://localhost:8080/api?timeout=32s": dial tcp 127.0.0.1:8080: connect: conn
ection refused
E0729 14:14:28.419533    90153 memcache.go:265] couldn't get current server API group li
st: Get "http://localhost:8080/api?timeout=32s": dial tcp 127.0.0.1:8080: connect: conn
ection refused
The connection to the server localhost:8080 was refused - did you specify the right hos
t or port?
```


Configuring user access by copying `/etc/kubernetes/admin.conf` to `~/.kube/config` and setting the correct ownership for the non-root user (ruben)

```
ruben@control:~/project$ mkdir -p ~/.kube
sudo cp -i /etc/kubernetes/admin.conf ~/.kube/config
sudo chown $(id -u):$(id -g) ~/.kube/config
```

Successfully retrieved node statuses via `kubectl get nodes -o wide`. All 3 nodes (control, node01, and node02) are registered in the cluster. (Note: Nodes are in *NotReady* state pending CNI network plugin installation)

```
ruben@control:~/project$ kubectl get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE
control	NotReady	control-plane	4m41s	v1.30.14	10.10.10.89	<none>	Ubu
node01	NotReady	<none>	68s	v1.30.14	10.10.10.40	<none>	Ubu
node02	NotReady	<none>	68s	v1.30.14	10.10.10.4	<none>	Ubu

2.3. CNI (Container Network Interface) Deployment

Since the cluster was initialized skipping kube-proxy, Cilium is deployed as the CNI plugin to manage pod networking and eBPF-based service routing.

```
ruben@control:~/project$ CILIUM_CLI_VERSION=$(curl -s https://raw.githubusercontent.com/cilium/cilium-cli/main/stable.txt)
curl -L --fail --remote-name-all https://github.com/cilium/cilium-cli/releases/download/${CILIUM_CLI_VERSION}/cilium-linux-amd64.tar.gz
sudo tar xzvfC cilium-linux-amd64.tar.gz /usr/local/bin
rm cilium-linux-amd64.tar.gz
```

% Total	% Received	% Xferd	Average Speed	Time	Time	Time	Current
			Dload Upload	Total	Spent	Left	Speed
0	0	0	0	0	0	0	0
100	68.75M	100	68.75M	0	16.01M	0	00:04

```
[sudo: authenticate] Password:
cilium
ruben@control:~/project$
```

```
ruben@control:~/project$ cilium install --set kubeProxyReplacement=true
[?] Using Cilium version 1.19.5
* Auto-detected cluster name: kubernetes
* Auto-detected kube-proxy has not been installed
[?] Cilium will fully replace all functionalities of kube-proxy
ruben@control:~/project$
```

Executing `kubectl get nodes -o wide` confirms that all nodes (control, node01, and node02) have transitioned from NotReady to Ready status, validating that pod network connectivity and eBPF routing are fully operational across the cluster.

```

ruben@control:~/project$ kubectl get nodes -o wide

```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE
control	Ready	control-plane	168m	v1.30.14	10.10.10.89	<none>	Ubuntu 26.04 LTS
node01	Ready	<none>	164m	v1.30.14	10.10.10.40	<none>	Ubuntu 26.04 LTS
node02	Ready	<none>	164m	v1.30.14	10.10.10.4	<none>	Ubuntu 26.04 LTS

Step 3: Cilium BGP Dynamic Routing & Edge Integration

3.1. OPNsense BGP Configuration (FRR Plugin):

To enable dynamic route propagation between the edge firewall and the Kubernetes cluster, FRRouting (FRR) is installed on gateway-01 (OPNsense). The firewall is configured as an Autonomous System (AS) to establish BGP peering sessions with the Kubernetes nodes (k8s-control-01, k8s-worker-01, and k8s-worker-02).

Install the Os-frr plugin in our Opnsense.

System: Firmware

Updates

The process will require 63 MiB more space.
12 MiB to be downloaded.

```

[1/13] Fetching net-snmp-5.9.5.2,1.pkg: ..... done
[2/13] Fetching libssh-0.12.0_1.pkg: ..... done
[3/13] Fetching frr10-10.6.1.pkg: ..... done
[4/13] Fetching os-frr-1.53_3.pkg: . done
[5/13] Fetching protobuf-c-1.5.1_4.pkg: ... done
[6/13] Fetching libyang3-3.13.6.pkg: ..... done
[7/13] Fetching abseil-20250127.1_1.pkg: ..... done
[8/13] Fetching jsoncpp-1.9.6_1.pkg: .. done
[9/13] Fetching libunwind-20250904.pkg: ... done
[10/13] Fetching rtrlib-0.8.0_1.pkg: . done
[11/13] Fetching c-ares-1.34.6.pkg: ... done
[12/13] Fetching protobuf-29.6,1.pkg: ..... done
[13/13] Fetching frr10-pythontools-10.6.1.pkg: . done
Checking integrity... done (0 conflicting)
[1/13] Installing abseil-20250127.1_1...
[1/13] Extracting abseil-20250127.1_1: ..... done
[2/13] Installing c-ares-1.34.6...

```

Output shown here for diagnostic purposes. There is no general need for manual system

Configure the BGP service.

OPNsense® securing networks made easy

root@OPNsense.internal

Lobby Reporting System Interfaces Firewall Routing General RIP OSPF OSPFv3 BGP BFD STATIC Diagnostics VPN Services Power Help

Routing: BGP

General Neighbors AS Path Lists Prefix Lists Community Lists Route Maps

Peer Groups

advanced mode full help

enable ☒

BGP AS Number 65000

BGP AD Distance

Router ID 10.10.10.1

Graceful Restart ☐

Network

Clear All Copy Paste Text

Bestpath Nothing selected

Clear All Select All

Network Import-Check ☒

Enforce First AS ☒

Log Neighbor Changes ☐

Maximum Paths

Maximum Paths (IBGP)

Route Redistribution ☐

Three BGP peers (k8s-control, k8s-Worker-01, and k8s-Worker-02) were created in OPNsense under FRR BGP configuration. Each node is assigned Remote AS 65001 matching its respective IP address on the local network (10.10.10.0/24)

OPNsense® securing networks made easy

root@OPNsense.internal

Lobby Reporting System Interfaces Firewall Routing General RIP OSPF OSPFv3 BGP BFD STATIC Diagnostics VPN Services Power Help

Routing: BGP

General Neighbors AS Path Lists Prefix Lists Community Lists Route Maps

Peer Groups

Search

50

Enabled	Description	Peer-IP	Remote AS	Prefix-Li...	Prefix-Li...	Commands
☒	k8s-control	10.10.10.89	65001			edit copy delete
☒	k8s-Worker-01	10.10.10.40	65001			edit copy delete
☒	k8s-Worker-02	10.10.10.4	65001			edit copy delete

Showing 1 to 3 of 3 entries

Apply

3.2. Cilium BGP Control Plane Configuration:

Make the yaml files to Kubernetes

```
ruben@control:~/project$ cat bgp-policy.yaml
apiVersion: cilium.io/v2alpha1
kind: CiliumBGPPeeringPolicy
metadata:
  name: bgp-peering-policy
spec:
  nodeSelector:
    matchLabels:
      kubernetes.io/os: linux
  virtualRouters:
  - localASN: 65001
    exportPodCIDR: false
    neighbors:
    - peerAddress: "10.10.10.1/32"
      peerASN: 65000
  serviceSelector:
    matchExpressions:
    - key: some-key
      operator: NotIn
      values:
      - never-match
ruben@control:~/project$
```

```
ruben@control:~/project$ cat ip-pool.yaml
apiVersion: cilium.io/v2alpha1
kind: CiliumLoadBalancerIPPool
metadata:
  name: external-pool
spec:
  blocks:
  - cidr: "10.10.10.200/29"
ruben@control:~/project$
```

Next, we put the next command for upgrade cillum configuration.

```
ruben@control:~/project$ cilium upgrade --set bgpControlPlane.enabled=true
[+] Using Cilium version 1.19.5
* Auto-detected cluster name: kubernetes
* Auto-detected kube-proxy has not been installed
[+] Cilium will fully replace all functionalities of kube-proxy
△ You maybe need to restart Cilium pods for configmap changes to take effect
```

Restart the Kubernetes service.

```
ruben@control:~/project$ kubectl rollout restart daemonset cilium -n kube-system
daemonset.apps/cilium restarted
ruben@control:~/project$ kubectl rollout restart deployment cilium-operator -n kube-system
deployment.apps/cilium-operator restarted
ruben@control:~/project$
```

```
ruben@control:~/project$ kubectl get pods -n kube-system -l k8s-app=cilium
NAME          READY   STATUS    RESTARTS   AGE
cilium-98wsq  1/1     Running   0           39s
cilium-bz9lz  1/1     Running   0           39s
cilium-hq2td  1/1     Running   0           22s
ruben@control:~/project$
```

HA_K8s_project © 2026 by R.Rubén is licensed under CC BY-NC-ND 4.0



Create the peer config yaml file for our infrastructure HA of Kubernetes.

```
ruben@control:~/project$ cat <<EOF > bgp-peer-config.yaml
apiVersion: cilium.io/v2
kind: CiliumBGPPeerConfig
metadata:
  name: cilium-bgp-peer-config
spec:
  gracefulRestart:
    enabled: true
EOF
ruben@control:~/project$ cat <<EOF > bgp-policy.yaml
apiVersion: cilium.io/v2
kind: CiliumBGPClusterConfig
metadata:
  name: cilium-bgp-cluster-config
spec:
  nodeSelector:
    matchLabels:
      kubernetes.io/os: linux
  bgpInstances:
    - name: "instance-65001"
      localASN: 65001
      peers:
        - name: "opnsense"
          peerASN: 65000
          peerAddress: "10.10.10.1"
          peerConfigRef:
            name: "cilium-bgp-peer-config"
EOF
```

```
ruben@control:~/project$ kubectl apply -f bgp-peer-config.yaml
kubectl apply -f bgp-policy.yaml
kubectl apply -f ip-pool.yaml
ciliumbgppeerconfig.cilium.io/cilium-bgp-peer-config created
ciliumbgpclusterconfig.cilium.io/cilium-bgp-cluster-config configured
Warning: cilium.io/v2alpha1 CiliumLoadBalancerIPPool is deprecated; use cilium.io/v2 C
iliumLoadBalancerIPPool
ciliumloadbalancerippool.cilium.io/external-pool unchanged
```

3.3. BGP Peering Verification:

Check the establishment of neighbour BGP

Verification of the BGP peering session between the Kubernetes cluster nodes and the OPNsense router using Cilium CLI. All three cluster nodes (control, node01, nodo02) successfully established BGP peering sessions with the upstream peer 10.10.10.1 (AS 65000).

```
ruben@control:~/project$ cilium bgp peers
Node      Local AS  Peer AS  Peer Address  Session State  Uptime  Family      Received  Advertised
control   65001     65000    10.10.10.1    established    2s      ipv6/unicast 0          0
          65001     65000    10.10.10.1    established    2s      ipv4/unicast 0          0
node01    65001     65000    10.10.10.1    established    2s      ipv6/unicast 0          0
          65001     65000    10.10.10.1    established    2s      ipv4/unicast 0          0
nodo02    65001     65000    10.10.10.1    established    1m22s   ipv6/unicast 0          0
          65001     65000    10.10.10.1    established    1m22s   ipv4/unicast 0          0
ruben@control:~/project$
```

Step 4. Deploying Test Application & LoadBalancer IP Announcement

Deployment of a sample application (Nginx) exposed via a Kubernetes Service of type LoadBalancer. Cilium dynamically allocates an IP address from the configured CiliumLoadBalancerIPPool (10.10.10.200/29) and announces the /32 route to OPNsense via BGP. Finally, external accessibility to the service is verified.

To validate the operational readiness of the BGP integration, a sample Nginx web application was deployed and exposed through a Kubernetes Service of type LoadBalancer. Cilium dynamically allocated an external IP (10.10.10.200) from the configured CiliumLoadBalancerIPPool (10.10.10.200/29) and advertised the prefix to the upstream peer (10.10.10.1 - OPNsense).

```
ruben@control:~/project$ kubectl get svc demo-nginx-svc
NAME                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
demo-nginx-svc      LoadBalancer        10.100.187.62    10.10.10.200     80:30212/TCP     24m
ruben@control:~/project$
```

```
ruben@control:~/project$ curl http://10.10.10.200
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```


Here, we can watch as Opnsense detected our High Availability cluster.

Routing: Diagnostics: BGP

> routerId : 10.10.10.1, localAS : 65000

IPv4 Routing Table

IPv6 Routing Table

Neighbors 

Summary

search

+

 10.10.10.4

-

 10.10.10.40

neighborAddr : 10.10.10.40

remoteAs : 65001

localAs : 65000

localAsReplaceAsDualAs : false

nbrExternalLink : true

localRole : undefined

remoteRole : undefined

nbrDesc : k8s-Worker-01

hostname : node01

bgpVersion : 4

remoteRouterId : 10.10.10.40

localRouterId : 10.10.10.1

bgpState : Established

bgpTimerUpMsec : 162000

bgpTimerUpString : 00:02:42

bgpTimerUpEstablishedEpoch : 1785413334

bgpTimerLastRead : 12000

bgpTimerLastWrite : 12000

CONCLUSION:

This project successfully demonstrates the deployment of a resilient, enterprise-grade Kubernetes infrastructure built on dynamic BGP routing and eBPF-based networking. By configuring Cilium in `kube-proxy` replacement mode and peering directly with an OPNsense edge firewall, external LoadBalancer services are automatically announced and routed across the cluster without manual network intervention

The integration of Ansible automation alongside Cilium's advanced networking features establishes a highly scalable, zero-downtime foundation ready for modern production workloads.